

# TE-SDF(Tetra-Encoded Signed Distance Field) 학습 자료

본 자료는 2026년 논문 “TE-SDF: Tetra-Encoded Signed Distance Field for Memory-Efficient and Accurate Collision Detection”를 바탕으로, 핵심 개념과 수식 전개를 바닥부터 설명합니다. 특히 접촉(컨택) 부분의 알고리즘을 이해하기 위해 필요한 배경 이론부터 실제 구현 단계까지 자세히 다루었습니다.

## 1. 서론

**Signed Distance Field (SDF)**는 공간의 각 점에서 가장 가까운 표면까지의 거리를 부호와 함께 저장하는 함수입니다. SDF 기반의 충돌 판정은 복잡한 기하를 견고하게 처리할 수 있기 때문에 물리 시뮬레이션, 게임, 애니메이션 및 로봇틱스에서 널리 사용됩니다.

그러나 전통적인 **격자(voxel) 기반 SDF**는 정확도를 높이기 위해 높은 해상도의 격자를 사용해야 하므로 메모리 요구량이 매우 커집니다. 이를 완화하기 위해 옥트리나 다항식 근사, 신경망 기반 SDF가 제안되었지만, 트리 탐색이나 네트워크 추론 과정이 복잡하여 Face-SDF 방식의 연속적인 SDF 평가에는 적합하지 않습니다.

논문에서는 이러한 한계를 극복하기 위해 **Tetra-Encoded Signed Distance Field (TE-SDF, TES)**를 제안합니다. TES는 **테트라헤드럴 메시(tetramesh)**의 적응형 공간 분할과 테트라 내부의 정확한 SDF 평가를 결합한 표현으로, GPU에서 효율적으로 충돌을 감지할 수 있는 것이 특징입니다.

## 2. 기본 개념 및 정의

### 2.1 테트라헤드럴 메시(tetramesh)

입체 표면을 삼각형으로 표현한 후, 볼륨 영역을 **테트라헤드럴 메시**(테트라 메시)로 분할할 수 있습니다. 논문에서는 삼각형 면들로 이루어진 표면(mesh)을 테트라 메시로 변환한 후, 노드 집합  $V$ , 면 집합  $F$ , 테트라 집합  $T$ 로 구성된 튜플  $G = (V, F, T)$ 으로 정의합니다. 노드는 표면 노드  $V_S$ , 내부 노드  $V_I$ , 외부 노드  $V_O$ 로 나뉘며, 테트라는 내부, 경계-내부, 경계-외부, 외부 테트라로 분류됩니다.

### 2.2 Signed Distance Field (SDF)의 정의

테트라 메시  $G = (V, F, T)$ 에 대해 SDF는 테트라가 정의된 영역 안팎에서 다음과 같이 정의됩니다:

$$\phi_G(x) = \begin{cases} -d(x, F), & \text{if } x \in T_{BI} \cup T_I, \\ d(x, F), & \text{if } x \in T_{BO} \cup T_O \end{cases}$$

여기서  $d(x, F)$ 는 점  $x$ 에서 면 집합  $F$ 까지의 최소 거리이며, 음수 부호는 점이 내부에 있다는 것을 의미합니다. TE-SDF는 이 SDF를 정확하게 계산하려는 방식을 개선합니다.

### 3. TES의 핵심 아이디어

#### 3.1 기본 직관

TES는 두 가지 관찰에서 출발합니다:

1. **테트라 메시의 적응성:** 지오메트리가 복잡한 영역에는 많은 테트라를, 단순한 영역에는 적은 테트라를 배치할 수 있습니다. 이는 필요한 부분에서 해상도를 세밀하게 조정할 수 있음을 뜻합니다.
2. **SDF 평가의 지역화:** 임의의 점에서 SDF를 정확히 계산하려면 그 점과 가장 가까운 표면 삼각형(면) 몇 개만 알면 됩니다.

#### 3.2 TES 정의

각 테트라  $t \in T$ 에 대해, TES는 테트라 내부의 임의의 점에서 SDF 값을 정확하게 계산하기 위해 **해당 테트라에 관련된 후보 면 집합  $E_t$** 를 저장합니다. 이렇게 하면 점  $x \in t$ 에서 SDF는 몇 개의 **점-삼각형 거리 계산**만으로 구해집니다. 수식으로는 다음과 같이 표현됩니다:

$$\phi_G(x) = \begin{cases} -d(x, E_t), & t \in T_{BI} \cup T_I, \\ d(x, E_t), & t \in T_{BO} \cup T_O, \end{cases} \quad (4)$$

여기서  $d(x, E_t) = \min_{f \in E_t} d(x, f)$ 는 점과 후보 면 집합 사이의 최소 거리입니다. 이 식은 TES가 테트라별로 정의되며, 음수/양수 구간에 따라 부호가 결정된다는 것을 보여줍니다.

### 4. TES 인코딩 과정

테트라 별로 후보 면 집합  $E_t$ 를 구축하는 것이 TES의 핵심입니다. 테트라  $t$ 와 면  $f$ 의 관계를 정의하기 위해 **갭 함수  $g_f(x)$** 를 도입합니다:

$$g_f(x) := d(x, f) - d(x, F).$$

즉, 특정 점에서 면  $f$ 까지의 거리와 전체 면 집합  $F$ 까지의 거리 차이를 나타냅니다. 테트라 내부에서 이 갭 함수가 **0이 되는 점이 존재하면**, 면  $f$ 는 테트라  $t$ 에서 SDF 평가에 기여하는 후보 면으로 간주합니다. 이는 다음과 같은 **가능성 판단 문제**로 표현됩니다:

$$\text{Find } x \in \text{int}(t) \text{ such that } g_f(x) = 0. \quad (5)$$

이 문제를 해결하는 과정에서 주요 단계를 소개합니다.

#### 4.1 Lipschitz 가지치기

모든 면에 대해 (5)를 직접 풀기에는 너무 많은 계산이 필요합니다. 논문에서는 **Lipschitz pruning**을 사용하여 대부분의 면을 빠르게 제외합니다. 원 구  $B(x_0, R)$ 가 테트라 내부를 포함할 때, 다음 조건이 성립하면 면  $f$ 는  $E_t$ 에 포함되지 않습니다:

$$g_f(x_0) \geq 2R. \quad (6)$$

이때  $x_0$ 는 테트라 중심,  $R$ 은 테트라를 둘러싸는 공의 반지름입니다. 이 조건을 만족하는 면들은 단순한 거리 비교로 빠르게 제거할 수 있으므로, 이후 단계에서 검토해야 할 후보 면 집합을 대폭 줄일 수 있습니다.

## 4.2 GPU 기반 브루트 포스 방법

Lipschitz pruning 이후 남은 후보 면에 대해 (5)를 만족하는지를 정확히 판단하기 위해, 저자들은 샘플링 기반 브루트 포스 방식을 사용합니다. 테트라 내부를 크기  $\alpha$ 로 축소한 후, 균일한 격자 해상도  $Q$ 로 내부를 샘플링하여, 각 샘플 점  $x$ 에 대해 갭 함수  $g_f(x)$ 를 계산하고,  $g_f(x) \leq \varepsilon$ 를 만족하는 면  $f$ 를  $E_t$ 에 추가합니다. 이 과정은 GPU에서 병렬로 실행되어 많은 테트라에 대한 계산을 동시에 수행합니다.

샘플링 밀도  $Q$ 가 충분히 높으면, 진정한 후보 면을 놓치지 않도록 보장할 수 있다는 이론적 결과가 부록 B에 제시되어 있습니다.

## 4.3 거리 기반 순위 절단

TES는 메모리 효율과 GPU 성능을 위해 각 테트라에 대해 저장하는 후보 면 수를 제한합니다. 이를 위해 **거리 기반 순위 절단(distance-ordered truncation)** 방법을 제안합니다. 각 후보 면  $f \in E_t$ 에 대해, 테트라 내부에서 면과의 최소 거리를 다음과 같이 계산합니다:

$$d_f = \min_{x \in t, g_f(x)=0} d(x, f). \quad (7)$$

즉, 테트라 영역에서 면이 가장 가까워지는 점을 찾고 그 거리를 저장합니다. 그 다음, 모든 후보 면을 이 거리  $d_f$  기준으로 오름차순 정렬하여 최대  $E_{\max}$ 개만 남깁니다. 이는 SDF 정확도가 중요한 **표면 근처**의 면을 우선적으로 보존하는 방식입니다. 실제 구현에서는 브루트 포스 단계에서 계산한 거리 근사치를 이용하여 정렬합니다.

## 4.4 변형(Deformation) 대응

테트라 메시의 노드 위치가 변하더라도, TES는 변형된 메시에서 정확한 SDF 값을 유지하기 위해 경계 테트라의 후보 면 집합을 확장합니다. 변형이 발생하면 경계면 주변 후보 면들이 추가되므로, 충돌 감지 정확도가 떨어지지 않습니다.

# 5. TES를 이용한 충돌 감지 (컨택)

본 절에서는 TE-SDF를 이용해 충돌(컨택) 특성을 생성하는 방법을 설명합니다. TES 기반 충돌 감지는 크게 **Node-TES**와 **Face-TES** 두 가지 방식으로 나누어집니다.

## 5.1 Node-TES

Node-TES 방식은 다른 테트라 메시의 SDF를 각 노드에서 평가하여 충돌 여부를 판단합니다. 전체 절차는 다음과 같습니다:

1. **브로드 단계:** 모든 노드-테트라 쌍  $(v, t)$  중에서 겹치는 경우를 효율적으로 찾아내는 광범위 탐색을 수행합니다. 이는 GPU에서 빠르게 실행되는 Karras의 방법을 사용합니다.
2. **내로우 단계:** 각 겹치는 쌍  $(v, t)$ 에 대해 한 개의 GPU 쓰레드를 할당하고, 점  $v$ 에서 TES 수식 (4)를 기반으로 SDF 값과 그래디언트를 계산합니다. 그래디언트는 점-삼각형 거리 계산을 통해 구해집니다.

Node-TES 방식은 각 노드에 대해 **국지적 SDF 값과 방향(그래디언트)**를 얻을 수 있으므로 간단하지만, 모서리-모서리 접촉이나 큰 면을 포함하는 경우 충돌을 놓칠 수 있습니다.

## 5.2 Face-TES

Face-TES는 Face-SDF 기법을 TES에 맞게 확장한 것으로, **겹치는 삼각형 면과 테트라**에 대해 최적화 문제를 풀어 SDF 최소값과 접촉 위치를 찾습니다. 절차는 다음과 같습니다:

1. **브로드 단계:** 잠재적으로 겹치는 면-테트라 쌍을 찾습니다.
2. **중간 단계:** 면과 테트라의 교차 영역을 계산하여 최대 6개의 꼭짓점을 가진 볼록한 다각형  $P$ 를 만듭니다.
3. **내로우 단계:** 다각형  $P$  위에서 다음 최적화 문제를 해결합니다:

$$\min_{x \in P} \varphi_G(x) = \min_{x \in P} \begin{cases} -d(x, E_t), & t \in T_{BI} \cup T_I, \\ d(x, E_t), & t \in T_{BO} \cup T_O. \end{cases} \quad (9)$$

즉, 테트라 영역과 겹치는 다각형 내부에서 TES SDF를 최소화하여 접촉점을 찾습니다. 논문에서는 **Frank-Wolfe 알고리즘**을 사용해 이 문제를 반복적으로 해결합니다.

### GPU 워프 병렬화

단순하게는 위의 최적화 문제를 면-테트라 쌍마다 하나의 쓰레드로 처리할 수 있지만, 각 쓰레드가  $E_t$ 에 저장된 면 정보를 반복적으로 불러오기 때문에 GPU 레지스터를 과도하게 사용하여 성능이 떨어집니다. 이를 해결하기 위해, 논문은 **한 개의 워프(32개 쓰레드)**를 하나의 최적화 문제에 할당합니다. 워프 내 각 쓰레드는 서로 다른 면을 담당하여 데이터를 레지스터에 저장하고, **warp shuffle** 연산을 통해 거리 계산 결과를 공유합니다. 이렇게 하면 레지스터 압박이 줄어 들고, Face-TES 내로우 단계가 효율적으로 수행됩니다.

### 코사인 필터 (Cosine filter)

Face-TES는 여러 국소 영역에서 접촉 특성을 생성하지만, 때때로 **스푸리어스(spurious)한 접촉 특성**이 발생할 수 있습니다. 예를 들어, 두 면의 법선 방향과 반대 방향으로 그래디언트가 계산되어 실제 최소 변위(minimal translational distance)와 상반되는 접촉이 검출될 수 있습니다. 이를 방지하기 위해, 논문에서는 다음 조건을 만족하는 접촉 특성을 필터링합니다:

$$\nabla \varphi \cdot n > -\cos \theta, \quad (10)$$

여기서  $\nabla \varphi$ 는 SDF의 그래디언트(접촉 노말),  $n$ 은 면의 방향,  $\theta$ 는 허용 각도 한계입니다. 내적이  $-\cos \theta$ 보다 크면 두 방향이 서로 비슷한 쪽을 향하고 있다는 뜻이므로, 해당 접촉은 유효하지 않은 것으로 간주하고 제거합니다. 이 필터는 추가 GPU 커널 실행 없이 Face-TES의 반환 과정에 통합됩니다.

## 5.3 TES의 장점과 한계

TES 기반 충돌 감지는 다음과 같은 장점이 있습니다:

- **메모리 효율성:** 테트라별로 제한된 후보 면만 저장하므로, 고해상도 SDF를 유지하면서도 수십 MB 수준의 메모리만으로도 복잡한 객체를 다룰 수 있습니다.
- **정확한 접촉:** 테트라 내부에서 정확한 점-면 거리 계산을 수행하므로, 깊게 파고든 경우에도 의미 있는 접촉 특성을 생성합니다.
- **GPU 병렬성:** Node-TES와 Face-TES 모두 GPU에서 효율적으로 병렬화되어 여러 객체 간의 충돌을 빠르게 감지할 수 있습니다.

다만, Face-TES는 모든 겹치는 면-테트라 쌍에 대해 국소 최적화 문제를 풀기 때문에 **접촉 제약의 수가 증가**할 수 있으며, 단순한 Node-SDF 방법보다 복잡합니다. 또한 TES 구축 과정에서 사용되는 샘플링 해상도와 거리 절단 파라미터를 적절히 조절해야 성능과 정확도 사이에서 균형을 맞출 수 있습니다.

## 6. 실험과 결과 요약

논문은 TES 인코딩과 충돌 감지를 다양한 시나리오에서 평가했습니다. 전체적인 결과는 본 자료의 목표 범위를 넘어가므로 간단히 요약하면 다음과 같습니다:

- **Robo-assembly 시나리오**에서 세 개의 팩-인-홀(pegin-hole)과 여러 볼트-너트 조립을 포함한 복잡한 메커니즘을 시뮬레이션하여, TES 충돌 감지기가 정확하게 작동함을 보였습니다. 충돌 검출 시간은 몇 밀리초 수준으로, 실시간 시뮬레이션에 적합합니다.
- **메모리 사용량 비교**에서 TES는 동일한 정확도를 제공하면서도 기존 방법보다 메모리 사용량이 크게 감소했습니다.
- **안정성** 측면에서 TES는 깊은 관통 상황에서도 그래디언트 필터와 지역적 최적화 덕분에 안정적인 접촉 특성을 제공합니다.

## 7. 요약 및 학습 포인트

본 자료를 통해 TE-SDF의 구성 요소와 수식 전개, 그리고 충돌 감지 알고리즘을 이해할 수 있습니다. 핵심 요점을 정리하면 다음과 같습니다:

1. **SDF와 테트라 메시 정의**: SDF는 점에서 표면까지의 서명 있는 거리를 나타내며, TES는 테트라 메시지를 이용해 공간을 적응적으로 분할합니다.
2. **TES 수식 (4)**: 테트라 내부에서 SDF를 계산할 때, 부호와 후보 면 집합에 따라 거리를 계산하는 방식을 정의합니다.
3. **갭 함수와 후보 면 결정 (5)**: 면  $f$ 가 TES에 포함되는지 결정하기 위해 갭 함수를 사용하고, 0이 되는 점의 존재를 판정합니다.
4. **Lipschitz 가지치기 (6)**: 간단한 거리 비교만으로 대부분의 면을 제거하여 계산량을 줄입니다.
5. **거리 기반 절단 (7)**: 테트라별로 저장되는 면의 수를 제한하여 GPU 성능과 메모리 효율을 확보합니다.
6. **Node-TES와 Face-TES**: 노드 기반과 면 기반 두 가지 충돌 감지 방식으로 접촉을 처리하며, Face-TES는 Frank-Wolfe 최적화와 워프 병렬화로 효율을 높입니다.
7. **코사인 필터 (10)**: 스푸리어스한 접촉 특성을 제거하기 위해 SDF 그래디언트와 면 법선의 내적을 기준으로 필터링합니다.

TES는 메모리 효율성과 정확도를 동시에 달성하는 혁신적인 SDF 표현으로, GPU 기반 접촉 시뮬레이션에서 큰 잠재력을 보여줍니다. 본 자료를 통해 TES의 원리와 수식 전개를 이해하고, 특히 접촉 처리 방법을 깊게 학습할 수 있기를 바랍니다.